☁️ Cloud App — e-Mandor

> Dokumentasi ini disusun dan difinalisasi oleh **Lead QA & Documentation** berdasarkan kontribusi seluruh anggota tim.

Aplikasi **e-Mandor** adalah sistem informasi berbasis **Cloud Computing** dengan pendekatan **Progressive Web App (PWA)** yang dirancang untuk mendigitalisasi pencatatan hasil panen kelapa sawit di tingkat afdeling.

Sistem ini memungkinkan:

- Mandor melakukan input absensi, jumlah janjang, dan brondolan melalui perangkat seluler
- Krani/administrasi memantau laporan produksi harian secara real-time
- Sinkronisasi data dari mode offline ke cloud ketika jaringan tersedia

Dengan arsitektur cloud-native, e-Mandor meningkatkan efisiensi operasional, akurasi data, serta transparansi proses pengupahan.

👥 Identitas Tim

Nama	NIM	Peran
Adam Ibnu Ramadhan	10231003	Lead Backend
Adhyasta Firdaus	10231005	Lead CI/CD & Deployment
Adonia Azarya Tamalonggehe	10231007	Lead QA & Documentation
Alfian Fadillah Putra	10231009	Lead Frontend
Varrel Kaleb Ropard Pasaribu	10231089	Lead DevOps

📅 Roadmap Progress

Minggu	Fokus	Target Milestone	Status
1	Setup Environment & Hello World	Full-stack hello world berjalan lokal	☑ Selesai
2	Backend REST API + PostgreSQL	6 CRUD endpoint + `/stats` terhubung ke DB	☑ Selesai
3	Frontend React + UI Integration	UI CRUD lengkap terhubung ke backend	☑ Selesai
4	Full-Stack Integration & Auth	CORS, JWT, environment variables	☑ Selesai

Minggu	Fokus	Target Milestone	Status
5	Docker Fundamentals — Dockerfile, Image & Container	Backend berjalan dalam Docker container, image di Docker Hub	☑ Selesai
6–7	Docker Lanjutan & Docker Compose	`docker compose up` menjalankan semua service	☐
8	UTS Demo	Full-stack + Docker (dinilai)	☐
9–11	CI/CD Pipeline	Auto test + auto deploy via GitHub Actions	☐
12–14	Microservices & API Gateway	Nginx reverse proxy + multi-service	☐
15–16	Polish, Security & UAS	Production-ready, monitoring	☐

🏗 Architecture Overview

```
flowchart LR
    subgraph Client_Device["🌐 Client"]
        A[Mandor / User]
        B[React PWA\nlocalhost:5173]
        C[Service Worker]
        D[Local Storage / IndexedDB]
    end

    subgraph Cloud_Server["☁️ Cloud Server"]
        E[FastAPI REST API\nlocalhost:8000]
        F[Validation Layer\nPydantic Schemas]
        G[Business Logic\nCRUD Functions]
        H[(PostgreSQL\ncloudapp DB)]
    end

    subgraph Deployment["📦 Deployment"]
        I[Docker Container]
    end

    A --> B
    B --> C
    C --> D
    D -->|Offline Data| C
    C -->|Sync When Online| E
    E --> F --> G --> H
    E -.-> I
```

Arsitektur ini menerapkan pendekatan **client–server berbasis REST API** dengan dukungan mode offline pada sisi frontend dan penyimpanan terpusat di cloud database.

Alur request lengkap:

```
flowchart TD
    CLIENT[🌐 Browser / React] -->|HTTP Request| MAIN[main.py\nFastAPI Router]
    MAIN -->|Validasi input| SCHEMA[schemas.py\nPydantic Schema]
    SCHEMA -->|Data valid| CRUD[crud.py\nBusiness Logic]
    CRUD -->|SQL via ORM| MODEL[models.py\nSQLAlchemy Model]
    MODEL --> DB[(PostgreSQL)]

    DB -.->|Result| MODEL
    MODEL -.->|Python Object| CRUD
    CRUD -.->|Response Object| SCHEMA
    SCHEMA -.->|JSON| MAIN
    MAIN -.->|HTTP Response| CLIENT
```

🛠️ Tech Stack

Kategori	Teknologi	Versi	Fungsi
Backend	Python + FastAPI	3.10+ / 0.115.0	REST API server
Frontend	React + Vite	18+	User interface (SPA/PWA)
Styling	Tailwind CSS	—	Utility-first CSS framework
Database	PostgreSQL	14+	Penyimpanan data relasional
ORM	SQLAlchemy	2.0.35	Python ↔ SQL mapping
Validation	Pydantic	2.9.0	Schema validasi request/response
Server	Uvicorn	0.30.0	ASGI server untuk FastAPI
Container	Docker & Docker Compose	—	Packaging & orchestration *(minggu 5–7)*
CI/CD	GitHub Actions	—	Automated test & deploy *(minggu 9–11)*

🐳 Docker Setup

Proyek ini menggunakan Docker untuk containerization. Backend dan frontend masing-masing memiliki Dockerfile yang dioptimalkan.

Prerequisites

- Docker Desktop terinstall dan running
- Akun Docker Hub (untuk push image)

Quick Start dengan Docker

1. Build Images

```
# Windows (Command Prompt/PowerShell)
scripts\docker.bat build

# Linux/Mac/WSL (Bash)
./scripts/docker.sh build
```

2. Run Containers

```
# Windows
scripts\docker.bat run

# Linux/Mac/WSL
./scripts/docker.sh run
```

Aplikasi akan berjalan di:

- Backend: http://localhost:8000
- Frontend: http://localhost:3000

3. Push ke Docker Hub

```
# Set username Docker Hub
set DOCKER_USERNAME=yourusername  # Windows
export DOCKER_USERNAME=yourusername  # Linux/Mac

# Push images
scripts\docker.bat push  # Windows
./scripts/docker.sh push  # Linux/Mac
```

Available Commands

Command	Description
`build`	Build backend dan frontend images
`run`	Run backend dan frontend containers
`push`	Push images ke Docker Hub
`clean`	Stop containers, remove images, cleanup
`status`	Show container dan image status
`logs-backend`	Show backend container logs

Command	Description
`logs-frontend`	Show frontend container logs

Manual Docker Commands

Jika ingin menggunakan Docker langsung:

```
# Build backend
cd backend && docker build -t yourusername/cloudapp-backend:v1 .

# Build frontend
cd frontend && docker build -t yourusername/cloudapp-frontend:v1 .

# Run backend
docker run -d -p 8000:8000 --env-file backend/.env yourusername/cloudapp-
backend:v1

# Run frontend
docker run -d -p 3000:80 yourusername/cloudapp-frontend:v1
```

Troubleshooting

- **Port conflict**: Pastikan port 8000 dan 3000 tidak digunakan aplikasi lain
- **Database connection**: Update `backend/.env` dengan `DATABASE_URL` yang benar untuk Docker
- **Permission denied**: Pastikan Docker Desktop running dengan akses admin

---

🗀 Struktur Proyek

```
cc-kelompok-a-awit/
├── README.md                        ← Dokumentasi proyek (file ini)
├── .gitignore
├── setup.sh                         ← Script setup otomatis
│
├── backend/
│   ├── Dockerfile                   ← 🐳 Docker build instructions (Modul 5)
│   ├── .dockerignore                ← 🐳 Exclude files dari Docker image
│   ├── .env.docker                  ← 🐳 Environment vars untuk Docker
│   ├── main.py                      ← FastAPI app & semua endpoint
│   ├── auth.py                      ← JWT authentication utilities
│   ├── models.py                    ← SQLAlchemy model (tabel database)
│   ├── schemas.py                   ← Pydantic schemas (validasi
request/response)
│   ├── crud.py                      ← Fungsi CRUD (business logic)
│   ├── database.py                  ← Koneksi ke PostgreSQL
│   ├── requirements.txt             ← Dependencies Python
│   ├── .env                         ← ⚠ TIDAK di-commit (berisi kredensial
```

```
DB)
│   └── .env.example                ← Template konfigurasi (aman di-commit)
│
├── frontend/
│   ├── Dockerfile                  ← 🔨 Multi-stage build (Modul 5)
│   ├── .dockerignore               ← 🔨 Exclude files dari Docker image
│   ├── nginx.conf                  ← 🔨 Nginx config untuk serving React
build
│   ├── src/
│   │   ├── App.jsx                 ← Root component & state management
│   │   ├── App.css                 ← Global styles
│   │   ├── main.jsx                ← Entry point React
│   │   ├── components/
│   │   │   ├── Header.jsx          ← Judul & statistik + status API
│   │   │   ├── SearchBar.jsx       ← Input pencarian dengan clear
│   │   │   ├── ItemForm.jsx        ← Form create/edit item
│   │   │   ├── ItemList.jsx        ← Container grid daftar items
│   │   │   ├── ItemCard.jsx        ← Card per item dengan Edit/Delete
│   │   │   ├── LoginPage.jsx       ← Halaman login/register
│   │   │   └── Notification.jsx    ← Komponen notifikasi
│   │   └── services/
│   │       └── api.js              ← Semua fungsi fetch API (service layer)
│   ├── public/
│   ├── package.json
│   └── vite.config.js
│
├── scripts/
│   └── docker-run.sh               ← 🔨 Script untuk menjalankan semua
container
│
├── docs/
│   ├── api-test-results.md         ← Hasil testing endpoint API (Modul 2)
│   ├── ui-test-results.md          ← Hasil testing UI CRUD (Modul 3)
│   ├── auth-test-results.md        ← Hasil testing JWT Auth (Modul 4)
│   ├── image-comparison.md         ← 🔨 Perbandingan ukuran Docker image
(Modul 5)
│   ├── setup-guide.md              ← Panduan setup lengkap
│   ├── database-schema.md          ← Schema database
│   ├── member-[Adam Ibnu Ramadhan].md
│   ├── member-[Adhyasta Firdaus].md
│   ├── member-[Adonia Azarya Tamalonggehe].md
│   ├── member-[Alfian Fadillah Putra].md
│   └── member-[Varrel Kaleb Ropard Pasaribu].md
│
├── image_w3/                       ← Screenshot UI testing Week 3
└── image_w4/                       ← Screenshot Auth testing Week 4
```

🚀 Getting Started

Prerequisites

Tool	Versi Minimum	Cek Instalasi
Python	3.10+	`python --version`
Node.js	18+	`node --version`
npm	9+	`npm --version`
Git	terbaru	`git --version`
PostgreSQL	14+	`psql --version`
Docker Desktop	terbaru	`docker --version`

---

1 Clone Repository

```
git clone https://github.com/aidilsaputrakirsan-classroom/cc-kelompok-a-
awit.git
cd cc-kelompok-a-awit
```

---

2 Setup Backend

```
cd backend

# Buat virtual environment
python -m venv venv

# Aktifkan (Windows)
venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Salin template konfigurasi
copy .env.example .env

# Edit .env — sesuaikan DATABASE_URL dengan konfigurasi PostgreSQL lokal Anda:
# DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/cloudapp
```

Buat database di PostgreSQL terlebih dahulu:

```
CREATE DATABASE cloudapp;
```

Jalankan backend:

```
uvicorn main:app --reload --port 8000
```

Backend tersedia di:

URL	Keterangan
`http://localhost:8000`	Base API
`http://localhost:8000/docs`	Swagger UI (dokumentasi interaktif)
`http://localhost:8000/health`	Health check

Atau gunakan script otomatis dari root direktori:

```
bash setup.sh
```

---

3 Setup Frontend

```
cd frontend
npm install
npm run dev
```

Frontend tersedia di: `http://localhost:5173`

> ⚠ Pastikan backend sudah berjalan sebelum membuka frontend agar status "API Connected" muncul di header.

---

4 Menjalankan via Docker (Opsional — Modul 5+)

Jika Docker Desktop sudah terinstall, backend bisa dijalankan dalam container:

```
# Build Docker image
cd backend
docker build -t cloudapp-backend:v1 .

# Jalankan container
docker run -d -p 8000:8000 --env-file .env.docker --name backend cloudapp-
backend:v1
```

Backend tersedia di: `http://localhost:8000`

Atau gunakan script untuk menjalankan semua service sekaligus:

```
# Dari root directory
bash scripts/docker-run.sh start    # Start all containers
bash scripts/docker-run.sh stop     # Stop all containers
bash scripts/docker-run.sh status   # Cek status containers
bash scripts/docker-run.sh logs     # Lihat logs
```

Service	URL	Keterangan
Frontend	`http://localhost:3000`	React app (nginx)
Backend	`http://localhost:8000`	FastAPI REST API
Database	`localhost:5433`	PostgreSQL

> 💡 **Catatan:** Saat menjalankan backend di container, gunakan `host.docker.internal` sebagai database host di `.env`, bukan `localhost`. File `.env.docker` sudah dikonfigurasi untuk ini.

📡 API Reference

Base URL

```
http://localhost:8000
```

Ringkasan Endpoint

No	Method	Endpoint	Auth	Deskripsi	Status Code
1	`GET`	`/health`	❌	Cek status API	200
2	`GET`	`/team`	❌	Informasi tim	200
3	`POST`	`/auth/register`	❌	Registrasi user baru	201 / 400 / 422
4	`POST`	`/auth/login`	❌	Login & dapatkan JWT token	200 / 401
5	`GET`	`/auth/me`	☑ Bearer	Profil user yang sedang login	200 / 401
6	`POST`	`/items`	☑ Bearer	Buat item baru	201 / 401 / 422
7	`GET`	`/items`	☑ Bearer	Ambil semua item (pagination & search)	200 / 401
8	`GET`	`/items/{item_id}`	☑ Bearer	Ambil satu item berdasarkan ID	200 / 401 / 404

No	Method	Endpoint	Auth	Deskripsi	Status Code
9	`PUT`	`/items/{item_id}`	☑ Bearer	Update item (partial update)	200 / 401 / 404
10	`DELETE`	`/items/{item_id}`	☑ Bearer	Hapus item	204 / 401 / 404
11	`GET`	`/items/stats`	☑ Bearer	Statistik inventory	200 / 401

---

1. GET `/health` — Health Check

Mengecek apakah server API sedang berjalan.

```
GET http://localhost:8000/health
```

Response `200 OK`:

```
{
  "status": "healthy",
  "version": "0.2.0"
}
```

---

2. GET `/team` — Team Information

Menampilkan informasi seluruh anggota tim.

```
GET http://localhost:8000/team
```

Response `200 OK`:

```
{
  "team": "cloud-team-a-awit",
  "members": [
    { "name": "Adam Ibnu Ramadhan", "nim": "10231003", "role": "Lead Backend"
},
    { "name": "Adhyasta Firdaus", "nim": "10231005", "role": "Lead CI/CD &
Deployment" },
    { "name": "Adonia Azarya Tamalonggehe", "nim": "10231007", "role": "Lead QA
& Documentation" },
    { "name": "Alfian Fadillah Putra", "nim": "10231009", "role": "Lead
Frontend" },
```

```
    { "name": "Varrel Kaleb Ropard Pasaribu", "nim": "10231089", "role": "Lead DevOps" }
  ]
}
```

---

3. POST `/items` — Create Item

Membuat item baru dan menyimpannya ke database.

```
POST http://localhost:8000/items
Content-Type: application/json
```

Request Body:

Field	Type	Required	Validasi	Deskripsi
`name`	string	☑ Ya	min 1, max 100 karakter	Nama item
`price`	float	☑ Ya	harus > 0	Harga item
`description`	string	❌ Tidak	—	Deskripsi item
`quantity`	integer	❌ Tidak	default `0`, tidak negatif	Jumlah stok

Request Body Example:

```
{
  "name": "Laptop",
  "price": 15000000,
  "description": "Laptop untuk cloud computing",
  "quantity": 5
}
```

Response `201 Created`:

```
{
  "id": 1,
  "name": "Laptop",
  "description": "Laptop untuk cloud computing",
  "price": 15000000,
  "quantity": 5,
  "created_at": "2026-03-07T10:30:00+08:00",
  "updated_at": null
}
```

Error Response `422 Unprocessable Entity` — validasi gagal (misal `price` negatif):

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "price"],
      "msg": "Input should be greater than 0",
      "input": -5000
    }
  ]
}
```

---

4. GET `/items` — List Items

Mengambil daftar semua item dengan dukungan pagination dan pencarian.

```
GET http://localhost:8000/items?skip=0&limit=20&search=
```

Query Parameters:

Parameter	Type	Default	Batas	Deskripsi
`skip`	integer	`0`	≥ 0	Offset untuk pagination
`limit`	integer	`20`	1 – 100	Jumlah item per halaman
`search`	string	—	—	Kata kunci (nama atau deskripsi)

Response `200 OK`**:**

```
{
  "total": 2,
  "items": [
    {
      "id": 2,
      "name": "Mouse Wireless",
      "description": "Mouse bluetooth",
      "price": 250000,
      "quantity": 20,
      "created_at": "2026-03-07T10:35:00+08:00",
      "updated_at": null
    },
    {
      "id": 1,
      "name": "Laptop",
      "description": "Laptop untuk cloud computing",
```

```
      "price": 15000000,
      "quantity": 5,
      "created_at": "2026-03-07T10:30:00+08:00",
      "updated_at": null
    }
  ]
}
```

> **Catatan:** Item diurutkan berdasarkan `created_at` terbaru terlebih dahulu (descending).

---

5. GET `/items/{item_id}` — Get Item by ID

Mengambil detail satu item berdasarkan ID.

```
GET http://localhost:8000/items/1
```

Parameter	Type	Deskripsi
`item_id`	integer	ID unik item di database

Response `200 OK`:

```
{
  "id": 1,
  "name": "Laptop",
  "description": "Laptop untuk cloud computing",
  "price": 15000000,
  "quantity": 5,
  "created_at": "2026-03-07T10:30:00+08:00",
  "updated_at": null
}
```

Error Response `404 Not Found`:

```
{
  "detail": "Item dengan id=1 tidak ditemukan"
}
```

---

6. PUT `/items/{item_id}` — Update Item

Memperbarui sebagian atau seluruh field item berdasarkan ID (**partial update** — hanya field yang dikirim yang diperbarui).

```
PUT http://localhost:8000/items/1
Content-Type: application/json
```

Request Body — semua field opsional:

Field	Type	Validasi	Deskripsi
`name`	string	min 1, max 100 karakter	Nama item
`price`	float	harus > 0	Harga item
`description`	string	—	Deskripsi item
`quantity`	integer	tidak negatif	Jumlah stok

Request Body Example — hanya update harga:

```
{
  "price": 14000000
}
```

Response `200 OK`:

```
{
  "id": 1,
  "name": "Laptop",
  "description": "Laptop untuk cloud computing",
  "price": 14000000,
  "quantity": 5,
  "created_at": "2026-03-07T10:30:00+08:00",
  "updated_at": "2026-03-07T11:00:00+08:00"
}
```

Error Response `404 Not Found`:

```
{
  "detail": "Item dengan id=1 tidak ditemukan"
}
```

---

7. DELETE `/items/{item_id}` — Delete Item

Menghapus item secara permanen dari database.

```
DELETE http://localhost:8000/items/1
```

Response `204 No Content` — tidak ada response body. Status `204` menandakan item berhasil dihapus.

Error Response `404 Not Found`:

```
{
  "detail": "Item dengan id=1 tidak ditemukan"
}
```

---

8. GET `/items/stats` — Item Statistics

Mengembalikan statistik ringkasan dari seluruh data inventory.

```
GET http://localhost:8000/items/stats
```

Response `200 OK` — inventory memiliki data:

```
{
  "total_items": 3,
  "total_value": 86350000,
  "most_expensive": {
    "name": "Laptop",
    "price": 15000000
  },
  "cheapest": {
    "name": "Mouse Wireless",
    "price": 250000
  }
}
```

Response `200 OK` — inventory kosong:

```
{
  "total_items": 0,
  "total_value": 0,
  "most_expensive": null,
  "cheapest": null
}
```

> **Catatan:** `total_value` dihitung dari `price × quantity` untuk setiap item, lalu dijumlahkan.

📝 Progress Pengerjaan per Minggu

Week 1 — Setup Environment & Hello World

Capaian:

- Pembentukan tim dan pembagian peran (Lead Backend, Frontend, DevOps, QA & Docs, CI/CD)
- Setup environment: Python 3.10+, Node.js 18+, Git, VS Code
- Join GitHub Classroom, buat repositori tim `cc-kelompok-a-await`
- Setup SSH key dan verifikasi koneksi ke GitHub
- Buat struktur folder proyek (`backend/`, `frontend/`, `docs/`)
- Buat `.gitignore` standar (Python, Node, Docker, IDE)
- Hello World Backend — FastAPI dengan endpoint `/`, `/health`, `/team`
- Hello World Frontend — React + Vite yang menampilkan data dari backend API
- Setiap anggota berhasil push commit pertama (`docs/member-[NAMA].md`)

Deliverable:

- Repository tim aktif dengan struktur folder
- Backend FastAPI berjalan di `localhost:8000`
- Frontend React berjalan di `localhost:5173` dan menampilkan response dari backend

Lead per area:

Tugas	Dikerjakan oleh
Struktur folder & `.gitignore`	Lead DevOps
Backend hello world + `/team` endpoint	Lead Backend
Frontend React hello world + fetch API	Lead Frontend
Identitas tim & peer review README	Lead QA & Docs
Setup branch & deployment placeholder	Lead CI/CD

Week 2 — Backend REST API + PostgreSQL

Capaian:

- Setup database PostgreSQL lokal: buat database `cloudapp`
- Implementasi koneksi database via SQLAlchemy ORM (`database.py`)
- Desain model database tabel `items` (`models.py`) dengan kolom: `id`, `name`, `description`, `price`, `quantity`, `created_at`, `updated_at`
- Implementasi Pydantic schemas untuk validasi request/response (`schemas.py`): `ItemCreate`, `ItemUpdate`, `ItemResponse`, `ItemListResponse`
- Implementasi CRUD functions (`crud.py`): create, get_items (dengan pagination & search), get_item, update_item, delete_item
- Update `main.py` dengan 6 endpoint CRUD + endpoint `/items/stats` tambahan

- Konfigurasi file `.env` dan `.env.example` untuk environment variables
- Testing seluruh endpoint via Swagger UI (`/docs`)

Deliverable:

- 8 endpoint API berfungsi penuh (lihat tabel di atas)
- Data tersimpan persisten di PostgreSQL
- File `docs/api-test-results.md` — dokumentasi hasil testing semua endpoint

Lead per area:

Tugas	Dikerjakan oleh
`models.py`, `crud.py`, endpoint CRUD	Lead Backend
Testing API via Swagger, dokumentasi `api-test-results.md`	Lead QA & Docs
Setup PostgreSQL, konfigurasi `.env`	Lead DevOps
Memahami format response untuk persiapan frontend	Lead Frontend
Setup `.env.example` & environment variables	Lead CI/CD

Week 3 — Frontend React + UI CRUD Integration

Capaian:

- Pembuatan API service layer (`frontend/src/services/api.js`) — centralized fetch functions untuk semua operasi CRUD
- Implementasi komponen React terpisah (component architecture):
  - `Header.jsx` — menampilkan judul app, jumlah item, dan status koneksi API
  - `SearchBar.jsx` — input pencarian dengan tombol clear
  - `ItemForm.jsx` — form create/edit item dengan validasi frontend dan mode switch otomatis
  - `ItemList.jsx` — grid layout daftar item dengan loading state dan empty state
  - `ItemCard.jsx` — card per item dengan tombol Edit & Hapus
- Update `App.jsx` — root component dengan state management terpusat (items, editingItem, searchQuery, loading, isConnected)
- Implementasi semua CRUD handler: `handleSubmit`, `handleEdit`, `handleDelete`, `handleSearch`
- Pindahkan `API_URL` ke environment variable (`VITE_API_URL`) via `frontend/.env`
- Testing seluruh alur CRUD via browser (10 test case)

Deliverable:

- UI CRUD lengkap berjalan di `localhost:5173` terhubung ke backend
- File `docs/ui-test-results.md` — dokumentasi 10 test case UI dengan screenshot
- File `frontend/.env` dan `frontend/.env.example`

Lead per area:

Tugas	Dikerjakan oleh

Tugas	Dikerjakan oleh
Semua komponen React + `App.jsx`	Lead Frontend
Pastikan API berjalan & bantu debug response	Lead Backend
Setup `VITE_API_URL` env var, konfigurasi proxy	Lead DevOps
Testing 10 fitur CRUD via UI, dokumentasi `ui-test-results.md`	Lead QA & Docs
Implementasi komponen Notification/Toast	Lead CI/CD

Week 4 — Full-Stack Integration & JWT Auth

Capaian:

- Konfigurasi environment variables backend (`SECRET_KEY`, `ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `ALLOWED_ORIGINS`) via `backend/.env`
- Fix CORS: dari wildcard `*` ke whitelist origin spesifik menggunakan env var
- Implementasi JWT Authentication:
  - `backend/auth.py` — JWT utilities: create token, verify token, hash password, dependency `get_current_user`
  - `backend/models.py` — tambah model `User` (email, hashed_password, is_active)
  - `backend/schemas.py` — tambah auth schemas: `UserCreate`, `UserResponse`, `LoginRequest`, `TokenResponse`
  - `backend/crud.py` — tambah `create_user`, `authenticate_user`
  - `backend/main.py` — tambah endpoint `/auth/register`, `/auth/login`, `/auth/me`, semua `/items` endpoint dilindungi auth
- Implementasi Frontend Auth:
  - `frontend/src/components/LoginPage.jsx` — halaman login/register dengan tab switch
  - `frontend/src/components/Notification.jsx` — komponen notifikasi sukses/gagal
  - `frontend/src/services/api.js` — token management, auth headers, error handling 401
  - `frontend/src/App.jsx` — auth state, conditional rendering login vs main app
  - `frontend/src/components/Header.jsx` — tampilkan nama user & tombol logout
- Testing auth end-to-end: register, login, protected endpoints, logout

Deliverable:

- Auth flow berfungsi: register → login → JWT token → protected CRUD → logout
- File `docs/auth-test-results.md` — hasil testing 11 test case + 1 bug report
- `backend/.env.example` & `frontend/.env.example` di-commit sebagai template
- `docs/setup-guide.md` — panduan setup lengkap dari clone hingga running

Lead per area:

Tugas	Dikerjakan oleh
`auth.py`, `models.py`, `crud.py`, endpoint auth	Lead Backend
`LoginPage.jsx`, `Notification.jsx`, update `App.jsx`, `Header.jsx`	Lead Frontend

Tugas	Dikerjakan oleh
Konfigurasi CORS whitelist, env vars backend + frontend	Lead DevOps
Testing auth end-to-end, dokumentasi `auth-test-results.md`, update README	Lead QA & Docs
`docs/setup-guide.md`, `.env.example` lengkap	Lead CI/CD

Week 5 — Docker Fundamentals: Dockerfile, Image & Container

Capaian:

- Instalasi dan verifikasi Docker Desktop + login Docker Hub
- Pembuatan `backend/Dockerfile` menggunakan `python:3.12-slim` dengan optimasi layer caching
- Pembuatan `backend/.dockerignore` untuk exclude file sensitif dan tidak perlu
- Build Docker image: `cloudapp-backend:v1`
- Konfigurasi `backend/.env.docker` dengan `host.docker.internal` untuk akses DB dari container
- Implementasi non-root user (`appuser`) di Dockerfile untuk keamanan
- Pembuatan `frontend/Dockerfile` dengan multi-stage build (Node.js build → Nginx serve)
- Pembuatan `frontend/nginx.conf` untuk serving React build
- Pembuatan `scripts/docker-run.sh` untuk menjalankan semua container secara manual
- Perbandingan ukuran image: `python:3.12` (362.7 MB) vs `slim` (41.18 MB) vs `alpine` (17.23 MB)

Deliverable:

- Backend berjalan dalam Docker container, bisa diakses via `http://localhost:8000`
- Dockerfile menggunakan best practices (slim base, layer caching, non-root user, .dockerignore)
- File `docs/image-comparison.md` — perbandingan ukuran Docker image Python
- README diperbarui dengan instruksi Docker build & run

Lead per area:

Tugas	Dikerjakan oleh
`Dockerfile`, `.dockerignore`, non-root user, push ke Docker Hub	Lead DevOps
Healthcheck di Dockerfile, pastikan app berjalan di container	Lead Backend
`frontend/Dockerfile`, `nginx.conf`, riset Docker	Lead Frontend
Perbandingan ukuran image `docs/image-comparison.md`, update README	Lead QA & Docs
`scripts/docker-run.sh`, `.env.docker`	Lead CI/CD

🔐 Authentication

Sejak **Modul 4**, seluruh endpoint `/items` dilindungi dengan **JWT (JSON Web Token)**. User harus register dan login terlebih dahulu untuk mendapatkan token sebelum bisa mengakses data.

Alur Autentikasi

```
Register → Login → Dapatkan Token → Kirim Token di setiap request
```

1. **Register** — `POST /auth/register` dengan `email`, `name`, `password` (min 8 karakter + special character)
2. **Login** — `POST /auth/login` dengan `email` & `password` → mendapatkan `access_token`
3. **Gunakan token** — sertakan di header setiap request:

   ```
   Authorization: Bearer <access_token>
   ```

4. **Logout** — hapus token dari state (client-side)

Contoh Login

```
POST http://localhost:8000/auth/login
Content-Type: application/json

{
  "email": "user@student.itk.ac.id",
  "password": "password123!"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiJ9...",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "email": "user@student.itk.ac.id",
    "name": "Nama User",
    "is_active": true,
    "created_at": "2026-03-22T09:00:00+08:00"
  }
}
```

> 💡 **Catatan:** Token berlaku selama 60 menit. Setelah expired, user perlu login kembali.

🧪 Testing & QA

API Testing (Week 2)

Testing dilakukan via **Swagger UI** (`http://localhost:8000/docs`).

Alur testing yang direkomendasikan:

```
flowchart TD
    START([🟢 Buka /docs]) --> POST["1️⃣ POST /items\nBuat 3 item berbeda"]
    POST --> GET_ALL["2️⃣ GET /items\nCek total: 3"]
    GET_ALL --> GET_ONE["3️⃣ GET /items/1\nCek item spesifik"]
    GET_ONE --> PUT["4️⃣ PUT /items/1\nUpdate harga saja"]
    PUT --> VERIFY["5️⃣ GET /items/1\nVerifikasi harga berubah"]
    VERIFY --> SEARCH["6️⃣ GET /items?search=laptop\nHarus muncul 1 item"]
    SEARCH --> STATS["7️⃣ GET /items/stats\nCek statistik inventory"]
    STATS --> DELETE["8️⃣ DELETE /items/1\nHarus response 204"]
    DELETE --> NOT_FOUND["9️⃣ GET /items/1\nHarus response 404"]
    NOT_FOUND --> END([✅ SEMUA PASS])

    style START fill:#70AD47,color:#fff
    style END fill:#70AD47,color:#fff
```

Expected Results:

Langkah	Endpoint	Expected Status	Expected Result
1	`POST /items` (×3)	`201 Created`	Item tersimpan dengan `id` unik
2	`GET /items`	`200 OK`	`total: 3`, 3 item di array
3	`GET /items/1`	`200 OK`	Data item id=1 lengkap
4	`PUT /items/1`	`200 OK`	Response berisi data terbaru
5	`GET /items/1`	`200 OK`	`price` berubah ke nilai baru
6	`GET /items?search=laptop`	`200 OK`	`total: 1`, hanya "Laptop"
7	`GET /items/stats`	`200 OK`	`total_items: 3`, `total_value` dihitung
8	`DELETE /items/1`	`204 No Content`	Tidak ada response body
9	`GET /items/1`	`404 Not Found`	`detail: "Item dengan id=1 tidak ditemukan"`

📄 Detail lengkap: `docs/api-test-results.md`

UI Testing (Week 3)

Testing dilakukan manual via browser (`http://localhost:5173`).

10 Test Case Alur CRUD:

No	Test Case	Komponen Diuji	Hasil
TC-01	Status koneksi API tampil "🟢 API Connected"	Header	☑ Pass
TC-02	Item dari database Modul 2 tampil di daftar	ItemList + GET /items	☑ Pass
TC-03	Tambah item baru via form	ItemForm + POST /items	☑ Pass
TC-04	Item baru langsung muncul di daftar	ItemList re-render	☑ Pass
TC-05	Klik Edit → form terisi otomatis data item	ItemForm mode Edit	☑ Pass
TC-06	Update harga dan klik Update	ItemForm + PUT /items/:id	☑ Pass
TC-07	Cari item via SearchBar	SearchBar + GET /items?search=	☑ Pass
TC-08	Klik Hapus → confirm dialog muncul	window.confirm()	☑ Pass
TC-09	Konfirmasi OK → item hilang dari daftar	DELETE /items/:id	☑ Pass
TC-10	Hapus semua → empty state 📭 tampil	ItemList empty state	☑ Pass

📄 Detail lengkap + screenshot: `docs/ui-test-results.md`

🐳 Containerization (Modul 5)

Sejak **Modul 5**, aplikasi backend sudah berjalan dalam Docker container.

Docker Image

Service	Base Image	Image Tag	Ukuran (Compressed)
Backend	`python:3.12-slim`	`cloudapp-backend:v1`	~71 MB (base + deps)
Frontend	`node` → `nginx:alpine`	`cloudapp-frontend:v1`	~30 MB

Mengapa `python:3.12-slim`?

Image	Compressed Size	Cocok untuk e-Mandor?
`python:3.12`	362.7 MB	❌ Terlalu besar untuk production
`python:3.12-slim`	41.18 MB	☑ **Pilihan terbaik** — glibc kompatibel, ukuran kecil
`python:3.12-alpine`	17.23 MB	❌ Banyak dependency butuh compile manual (psycopg2, bcrypt, dll)

📄 Detail lengkap: `docs/image-comparison.md`

Dockerfile Best Practices

Dockerfile backend mengimplementasikan:

- ☑ **Slim base image** — `python:3.12-slim` (~41 MB vs ~363 MB full)
- ☑ **Layer caching** — `COPY requirements.txt` sebelum `COPY . .` agar dependencies di-cache
- ☑ **Non-root user** — `USER appuser` untuk keamanan
- ☑ **`.dockerignore`** — exclude `.env`, `__pycache__`, `.git`, `venv/`
- ☑ **`--no-cache-dir`** — pip install tanpa cache untuk hemat space

Quick Commands

```
# Build backend image
cd backend
docker build -t cloudapp-backend:v1 .

# Jalankan container
docker run -d -p 8000:8000 --env-file .env.docker --name backend cloudapp-
backend:v1

# Lihat container berjalan
docker ps

# Lihat logs
docker logs backend

# Stop & hapus container
docker stop backend && docker rm backend
```

> 🚧 **Minggu 6–7:** Docker Compose akan memungkinkan seluruh service (backend + frontend + database) berjalan dalam satu network terisolasi:
>
> ```
> docker compose up
> ```

--> Update week 6: fix git email attribution

🚀 Deployment

> 🚧 Akan diimplementasikan pada **Minggu 9–11** (CI/CD) dan **Minggu 11** (Cloud Deployment).

Rencana deployment:

- **CI/CD:** GitHub Actions untuk automated test & build
- **Cloud Platform:** Railway atau Render (PaaS)
- **Database:** Railway PostgreSQL atau Supabase

☑ QA Checklist

Week 1

- ☑ Identitas tim lengkap & terdokumentasi
- ☑ Setiap anggota memiliki minimal 1 commit
- ☑ Backend hello world berjalan (`/health`, `/team`)
- ☑ Frontend React menampilkan data dari backend
- ☑ Struktur folder proyek rapi

Week 2

- ☑ Semua 8 endpoint API berfungsi dan terdokumentasi
- ☑ Request body & validasi terdokumentasi per endpoint
- ☑ Response schema konsisten dengan kode (`schemas.py`)
- ☑ Error response (404, 422) terdokumentasi
- ☑ Alur testing API terdokumentasi dengan expected results
- ☑ File `docs/api-test-results.md` dibuat dan lengkap

Week 3

- ☑ Semua operasi CRUD berfungsi via UI (Create, Read, Update, Delete)
- ☑ 10 test case UI lulus semua (☑ Pass)
- ☑ Komponen React terstruktur rapi (minimal 5 komponen terpisah)
- ☑ Environment variable (`VITE_API_URL`) dikonfigurasi
- ☑ Struktur proyek diperbarui di README
- ☑ File `docs/ui-test-results.md` dibuat dengan screenshot
- ☑ `frontend/.env.example` di-commit sebagai template

Week 4

- ☑ JWT Authentication berfungsi end-to-end (register → login → token → protected endpoint)
- ☑ CORS dikonfigurasi dengan whitelist (bukan wildcard `*`)
- ☑ Semua secrets tersimpan di `.env`, tidak ada hardcoded secret
- ☑ Login page muncul saat buka app
- ☑ Register → otomatis login berfungsi
- ☑ Nama user tampil di header + tombol logout
- ☑ Logout → kembali ke login page
- ☑ Data items persistent setelah logout dan login kembali
- ☑ Endpoint tanpa token → 401 Unauthorized (negative test pass)
- ☑ File `docs/auth-test-results.md` dibuat dengan 11 test case + bug report
- ☑ README diperbarui: section Authentication + endpoint list lengkap
- ☐ ~~Tambah/update item via UI~~ — Bug BUG-04-01 (assignee: Lead Frontend)

Week 5

- ☑ Docker Desktop terinstall dan berjalan
- ☑ `backend/Dockerfile` dibuat dengan best practices (slim base, layer caching, non-root user)
- ☑ `backend/.dockerignore` exclude file sensitif (.env, **pycache**, .git, venv)
- ☑ Backend berhasil berjalan dalam Docker container (`http://localhost:8000`)

- ☑ `frontend/Dockerfile` dibuat dengan multi-stage build
- ☑ `frontend/nginx.conf` dibuat untuk serving React build
- ☑ `scripts/docker-run.sh` dibuat untuk menjalankan semua container
- ☑ Perbandingan ukuran image `python:3.12` vs `slim` vs `alpine` terdokumentasi
- ☑ File `docs/image-comparison.md` dibuat — rekomendasi: `python:3.12-slim`
- ☑ README diperbarui: section Containerization + Docker instructions

---

📚 Dokumentasi Tambahan

Dokumen	Lokasi	Keterangan
API Test Results	`docs/api-test-results.md`	Hasil testing endpoint API — Modul 2
UI Test Results	`docs/ui-test-results.md`	Hasil testing UI CRUD — Modul 3
Auth Test Results	`docs/auth-test-results.md`	Hasil testing JWT Auth end-to-end — Modul 4
Image Comparison	`docs/image-comparison.md`	Perbandingan ukuran Docker image Python — Modul 5
Setup Guide	`docs/setup-guide.md`	Panduan setup dari clone hingga running
Database Schema	`docs/database-schema.md`	Skema tabel database
Swagger UI	`http://localhost:8000/docs`	Dokumentasi API interaktif (saat backend berjalan)

🔗 Links

- **Repository:** GitHub Classroom
- **Swagger UI:** `http://localhost:8000/docs` *(saat backend berjalan)*
- **Frontend:** `http://localhost:5173` *(saat frontend berjalan)*

---

*Dokumentasi ini dikelola oleh **Adonia Azarya Tamalonggehe** (Lead QA & Documentation).*
Institut Teknologi Kalimantan — Komputasi Awan 2026.